
90 minutes. Tous Documents papier autorisés. Les appareils électroniques ne sont pas autorisés.

Liste des exercices et Barème à titre indicatif.

Page

1 Entrelacement (3 pts)

Voici un pseudo-programme concurrent:

```
x := x + 1      ||      x := 1
x := x + 1      ||      x := 2 * x
```

1. En supposant que l'affectation est atomique, combien d'entrelacements différents sont possibles ?
2. Si x vaut 0 au démarrage du programme, quelles sont les valeurs possibles de x après son exécution? Donnez un entrelacement qui conduit à chacune de ces valeurs.
3. Mêmes questions si l'affectation n'est pas atomique, mais composée d'une lecture atomique et d'une écriture atomique.

2 FairThreads (6 pts)

Pour les élections, NB **bureaux de votes** sont ouverts. Chaque **votant**, qui se trouve initialement dans la **rue**, choisit un **bureau de vote** au hasard. Arrivé au bureau il distribue 5 voix, une par une, au hasard, entre NC candidats (il peut donner plusieurs - ou toutes ses - voix à un même candidat), les voix de chaque bureau sont stockées dans une **urne**. Elle sont récupérées régulièrement par des **assesseurs** et portées dans un **coffre** qui se situe dans une **administration**.

On représentera la **rue**, les **bureaux de vote** et l'**administration** par des ordonnanceurs, les **votants** et les **assesseurs** par des threads, et les **urnes** et les **coffres** par des tableaux (**urne[i]** est le nombre de voix du candidat numéro **i** dans **urne** et **coffre[i]** est le nombre de voix du candidat numéro **i** dans **coffre**).

On précise le comportement des threads:

- A la création les **votants** sont dans la **rue**. Au bout d'un certain temps ils choisissent aléatoirement un **bureau de vote** et (déçus par l'offre politique) distribuent leur 5 voix au hasard dans l'**urne** parmi les candidats présents. Entre chacun de leurs votes, ils laissent la possibilité à d'autres votants du même bureau d'accéder à l'**urne**. Quand ils ont distribué leur cinq voix, ils rentrent chez eux (ils disparaissent du système).
- Les **assesseurs** sont attachés à l'**administration** à la création. Régulièrement, ils s'attachent à un bureau de vote (au hasard) **qui ne contient pas déjà un assesseur**, ils collectent l'ensemble des voix de l'**urne** (la remettant "à zéro"), retournent à l'**administration** et ajoutent les voix collectées au **coffre**.

1. Faire un schéma décrivant une implémentation en FairThreads du système en faisant apparaître les différents threads, les structures de données, les ordonnanceurs et les actions de liaison-déliasion éventuelles.
2. Donner le code C pour les définitions des structures de données partagées et la création des ordonnanceurs.
3. Écrire une routine **votant** décrivant le comportement d'un **votant**. Préciser dans quel ordonnanceur les **votants** doivent être lancés.
4. Faire la même chose pour la routine **assesseur**.
5. On veut ajouter un système pour la cloture du **bureau de vote**. Un thread **président** décide, au bout d'un certain temps, que l'élection est close, il se déplace dans chaque **bureau de vote** pour le fermer (ce dernier ne peut plus accepter de voix) puis revient à l'**administration**, choisit un unique **assesseur** et l'envoie faire le tour des **bureaux de vote** pour récupérer les dernières voix dans les **urnes**. Enfin, il publie les résultats de l'élection sur la **sortie standard**.

Décrire les modifications à faire dans le schéma et dans le code pour inclure ce système de cloture de l'élection.

3 Canaux Synchrones en Go (7 pts)

On souhaite implémenter en Go une bibliothèque de fonctions des entiers dans les entiers. Le système doit fonctionner ainsi:

- les goroutines **clientes** peuvent **enregistrer** une fonction en communiquant avec la goroutine **bibliothèque** et en lui envoyant une fonction (des entiers dans les entiers) et un identifiant (une chaîne de caractères), ce qui associe la fonction à l'identifiant dans la **bibliothèque**. Si une fonction est déjà associée à cet identifiant, elle est écrasée.
Par exemple une **cliente** peut envoyer `func(x int){return x * x}` et `"carre"` pour enregistrer la fonction `"carre"` dans la **bibliothèque**.
- les goroutines **clientes** peuvent contacter la **bibliothèque** pour **appeler** une fonction enregistrée, en communiquant un identifiant et un argument entier. La **cliente** attend ensuite de recevoir le résultat du calcul.
Par exemple une **cliente** peut envoyer `"carre"` et 3 et recevoir, au bout d'un certain temps, le résultat 9.

Aide: Syntaxe des tables d'association en go:

```
map[TypeClef]TypeValeur          // type
m = make(map[TypeClef]TypeValeur) // creation
m[clef]                          // acces
m[clef] = valeur                 // insertion/remplacement
```

1. Faire un schéma du système en listant les différents canaux impliqués, avec leurs types.
2. Écrire la goroutine **bibliothèque**.
3. Écrire trois goroutines **clientes** qui enregistrent et utilisent des fonctions simples dans une **bibliothèque** initialement vide, de manière concurrente. Décrire les conséquences possibles des divers entrelacements.
4. Ajouter à la **bibliothèque** un mécanisme d'appel retardé (utilisant les **futurs**) : les **clientes** peuvent maintenant appeler la **bibliothèque** avec un entier et un identifiant et recevoir instantanément (sans que la **bibliothèque** prenne du temps à calculer) une *promesse* de résultat (un canal dédié). La **bibliothèque** lance simultanément le calcul (sur une goroutine dédiée). La **cliente** peut utiliser la *promesse* pour récupérer son calcul quand elle en a besoin.
Par exemple, la **cliente** envoie `"carre"` et 3 et reçoit instantanément une *promesse* `p`. Plus tard, elle peut contacter la **bibliothèque** pour recevoir le résultat promis par `p` et attendre de recevoir la valeur 9.
5. On voudrait déplacer la **bibliothèque** sur un serveur distant. Décrire un code client et serveur (ainsi que le protocole de communication) qui permet d'utiliser la **bibliothèque** à travers Internet, en accès direct uniquement (pas d'accès retardé et pas d'enregistrement de fonction).
6. Expliquer les difficultés rencontrées pour effectuer l'enregistrement de fonctions à travers Internet, proposer des solutions.

4 Canaux Synchrones en OCaml (5 pts)

Les questions A) et B) sont indépendantes.

- A) On souhaite implémenter un système composé d'une **pile** d'entiers et de deux **clients** qui font quelques *push* et *pop* concurrents.
- 1) Écrire une fonction **pile** destinée à être la routine d'un thread implémentant une pile (une LIFO) qu'on peut contacter sur deux canaux : **push** pour ajouter un élément à la pile, **pop** pour récupérer l'élément en tête de la pile, et l'enlever de la pile (**pop** sur une pile vide fait exploser le programme).
Le code de la pile devrait être polymorphe (à défaut, faire une pile d'entiers).
 - 2) Donner le code des deux **clients** qui font quelques **push** et **pop** concurrents.
- B) On souhaite implémenter un système composé d'un **diffuseur** qui envoie **n** copies d'un message **m** à **k** **receveurs** via une liste de **k** canaux **li** (un canal par **receveur**). Ces **n** envois ne doivent pas être bloquants si au moins une réception sur un des canaux de **li** est possible, même si une réception a déjà été faite sur ce canal. Un canal de **li** peut ne pas être utilisé (par exemple si la longueur de **li** est inférieure à **n**) ou être utilisé deux fois. Par exemple si **li** vaut [**c1**; **c2**; **c3**] et **n** vaut 3, le message peut être envoyé à **c3**, **c1** puis **c3** (si personne n'écoute sur **c2** et que **c3** et **c1** sont disponibles dans cet ordre).
- 1) Implémenter une fonction **recoit** destinée à être la routine d'un thread **ecouteur**.
 - 2) Le thread **diffuseur** est initialisé avec une liste **li** de canaux transportant du '**a**, un message **m** de type '**a** et un entier **n** et qui envoie **n** fois le message **m** sur des canaux disponibles de **li**. Implémenter la fonction **nenvoi** destinée à être la routine du **diffuseur**.

5 Modélisation (5 pts)

On modélise une banque de dépôt composée de **NB clients** et d'une **banque** qui communiquent via des canaux **depot** et **retrait**.

Les **clients** disposent d'une somme initiale variable (par exemple 100, 0 ou 500). Les **NB clients** (par exemple 3) peuvent choisir de déposer ou de retirer de l'argent en envoyant des valeurs sur les canaux **depot** et **retrait** et en attendant, à chaque fois, une validation (ou une invalidation) de l'opération.

La **banque** écoute sur ces canaux et maintient les comptes de chacun des **clients** (représentés par un tableau d'entiers **relatifs** de taille 3) ainsi que la somme totale d'argent dont elle dispose (initialisée à 1000). Elle envoie une validation au **client** après chaque opération. Elle refuse un retrait s'il ferait passer le compte du **client** en dessous de -500.

1. Faire un schéma du système.
2. Modéliser le système en Promela.
3. Ajouter un **observateur** au système qui garantit que la somme possédée par la **banque** est positive.
4. Donner une trace du système qui viole cette propriété.
5. Énoncer une propriété de vivacité (pertinente) pour ce système.
6. Ajouter un **observateur** qui vérifie que la somme totale de l'argent détenu à la fois par la **banque** et les **clients** est constante.